Объяснение последнего Colab: нейросеть XOR

Сверено с 20 ячейками файла XOR_Colab_простая_ООП.ipynb и главой 20 книги Криса Элбона

XOR: объяснение последней версии Colab

Подробный разбор каждой части кода, архитектур, обучения, графиков и материала книги Криса Элбона.

> **Это руководство относится именно к файлу `XOR_Colab_простая_ООП.ipynb` — 20 ячеек.** Номера ниже обозначают фактический порядок ячеек (включая Markdown и код), а не номера запуска вида `[1]`. Если ты перенёс ячейки или переписал их вручную, ориентируйся также на заголовок и начало кода.

Как открыть нужное место: в верхней части HTML-файла вставь **адрес своего открытого блокнота Google Colab** (начинается с `https://colab.research.google.com/`). После этого ссылки «Открыть ячейку» рядом с разделами будут вести к конкретным ячейкам через `#scrollTo=...`. Если адрес не вставлен, ищи по заголовкам и номерам, указанным здесь. Для PDF используй номера ячеек.

Карта блокнота: где что находится

Ячейка	Тип	Что в ней	Где разбираем
1	текст	Таблица истинности XOR	§1
2	код	NumPy, Keras, Matplotlib и данные	§2
3	текст	Идея архитектуры 2–4–1	§3
4	код	`build_model`, слои, `compile` и настройки	§3–4
5	текст	Что происходит при обучении	§5
6	код	`fit`, `predict`, `evaluate`, `learning_epochs`	§5–6
7	текст	Формулы MSE и BCE	§7
8	код	Ручной расчёт ошибок	§7
9	текст	Объяснение двух графиков	§8
10	код	Графики основной сети	§8
11	текст	План сравнения трёх архитектур	§9
12	код	Цикл экспериментов, графики и эпохи	§9
13	текст	Обоснование выбранных решений	§10
14	текст	Постановка задачи для ООП-сети	§11
15	код	Класс `NeuralNetwork`	§11–13
16	текст	Зачем создаём новый объект	§14
17	код	Запуск ООП-сети и результаты	§14
18	текст	Пояснение графиков ООП	§14
19	код	Графики собственной сети	§14
20	текст	Шпаргалка для защиты	§15

1. Что мы решаем: XOR

Ячейка 1.

Открыть таблицу XOR ↗

XOR (исключающее ИЛИ) даёт **1, если входы различаются**, и **0, если совпадают**:

x_1	x_2	Правильный y
0	0	0
0	1	1
1	0	1
1	1	0

Написать `int(x1 != x2)` было бы легко, но это **готовое правило**, а не машинное обучение. Мы даём сети примеры и ответы, а она сама подбирает веса и смещения. Главная педагогическая причина выбрать XOR: **одной прямой нельзя отделить две единицы от двух нулей**. Это называется линейной неразделимостью. Классы расположены на противоположных диагоналях квадрата.

Один выходной `Dense(1, activation="sigmoid")` без скрытого слоя задаёт линейную границу решения. Поэтому контрольная модель 2–1 не способна идеально реализовать XOR. Нелинейный скрытый слой меняет ситуацию.

Для преподавателя: «XOR — небольшая задача бинарной классификации, удобная для демонстрации необходимости скрытых нелинейных слоёв».

2. Импорты и обучающие данные

Ячейка 2.

Открыть подготовку данных ↗

```
import numpy as np
import matplotlib.pyplot as plt
import keras

np.set_printoptions(precision=5, suppress=True)

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=np.float32)
y = np.array([0, 1, 1, 0], dtype=np.float32)
```

- `numpy` (`np`) — числовые массивы и матричные операции. В ООП-реализации это единственная математическая библиотека.
- `matplotlib.pyplot` (`plt`) — графики Loss и Accuracy.
- `keras` — готовые слои и обучение для первой части работы.
- `np.set_printoptions(...)` **только оформляет вывод**, не меняет веса или обучение. `precision=5` — число отображаемых знаков, `suppress=True` — меньше научной записи.
- `X` имеет форму `(4, 2)`: четыре строки, два входных признака.
- `y` имеет форму `(4,)`: четыре правильные метки.
- `dtype=np.float32` даёт числовой формат, привычный для нейросетевых вычислений.

Следующие `print` показывают версию Keras, используемый backend и содержимое `X`, `y`. Они нужны для диагностики и понимания данных, а не для самого обучения.

Почему не стандартизируем? Оба признака уже принимают только 0 и 1 и имеют одинаковый масштаб. В разделе **20.1 книги** объясняется польза стандартизации, но отмечается, что она не всегда необходима для бинарных признаков.

3. Архитектура основной сети: 2–4–1

Ячейки 3–4.

Прочитать описание архитектуры ↗

.

Открыть код build_model ↗

Основная модель имеет **два входа → четыре скрытых нейрона `tanh` → один выходной нейрон `sigmoid`**. Это запись **2–4–1**.

```
def build_model(hidden_layers=(4,), activation="tanh", learning_rate=0.05):
    model = keras.Sequential(name="xor_network")
    model.add(keras.Input(shape=(2,)))
    for width in hidden_layers:
        model.add(keras.layers.Dense(width, activation=activation))
    model.add(keras.layers.Dense(1, activation="sigmoid"))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model
```

По строкам:

1. `def build_model(...)` — функция-фабрика: каждый её вызов создаёт **новую** сеть. Это пригодится при независимых экспериментах.
2. `hidden_layers=(4,)` — по умолчанию один скрытый слой с четырьмя нейронами. Запятая в `(4,)` означает кортеж с одним элементом. `()` означало бы вообще без скрытых слоёв; `(6, 4)` — два скрытых слоя.
3. `keras.Sequential` — слои выполняются последовательно.
4. `keras.Input(shape=(2,))` — один пример состоит из двух чисел. Это не дополнительный обучаемый слой.
5. `for width in hidden_layers` — строит столько скрытых `Dense`, сколько записано в кортеже. `Dense` — полносвязный слой, где каждый его нейрон получает выходы всех нейронов предыдущего слоя.
6. `Dense(1, sigmoid)` — один выход: вероятность класса 1; порог 0,5 превратит её в 0 или 1.
7. `compile(...)` — выбор алгоритма изменения весов, функции ошибки и метрики. `return model` отдаёт созданную модель.

Сколько параметров? Первый обучаемый слой: `2 × 4 + 4 = 12` (восемь весов и четыре смещения). Второй: `4 × 1 + 1 = 5`. **Итого 17 параметров.** Вес — коэффициент влияния входа; смещение (`bias`) помогает сдвигать границу решения. Сеть учится изменять их, а не исполняет заранее записанный `if`.

В **ячейке 4** параметры эксперимента заданы отдельно:

```
HIDDEN_LAYERS = (4,)
ACTIVATION = "tanh"
LEARNING_RATE = 0.05
TRAIN_EPOCHS = 500
```

`TRAIN_EPOCHS=500` — **запланированная длина обучения**, а вовсе не утверждение, что для решения XOR понадобилось именно 500 эпох.

4. Почему именно tanh, sigmoid, BCE и Adam

Ячейки 3–4; объяснения также в ячейке 13.

Открыть параметры модели ↗

.

Открыть обоснование ↗

Для отдельного нейрона сначала рассчитывается взвешенная сумма $z = x_1w_1 + x_2w_2 + b$, а затем применяется функция активации $f(z)$.

`tanh(z)` в скрытом слое: возвращает значения от −1 до 1. Эта функция нелинейная и имеет выходы по обе стороны нуля. Она позволяет сети построить нелинейную границу для XOR. Мы выбрали её как понятный рабочий вариант, а **не как доказанно лучший**.

`sigmoid(z) = 1/(1+exp(−z))` на выходе: выдаёт вероятность от 0 до 1. При $p \geq 0.5$ относим пример к классу 1, иначе к классу 0. Она нужна именно потому, что у задачи два возможных класса.

`ReLU(z) = max(0,z)` появляется в истории учебных примеров и может использоваться в иных экспериментах, но в **последней версии нашего блокнота её нет**: мы оставили только три экспериментальные архитектуры, среди которых скрытые слои используют `tanh` либо `sigmoid`.

`binary_crossentropy` (BCE) — функция ошибки для предсказанных вероятностей и правильных меток 0/1. Даже если класс уже правильный, BCE уменьшается по мере приближения вероятности к правильному 0 или 1.

`Adam(learning_rate=0.05)` — оптимизатор Keras. Он использует градиенты и адаптивно подбирает изменение параметров. `0.05` — достаточно крупный выбранный экспериментальный шаг; мы **не доказали**, что он оптимален. Важно: ООП-версия в ячейке 15 использует **не Adam**, а простой градиентный спуск с `lr=0.5`.

Почему несколько слоёв вообще помогают? Последовательность только линейных преобразований сводится к одному линейному. Нелинейные скрытые активации позволяют моделировать XOR, который линейно неразделим.

5. Обучение основной Keras-сети

Ячейки 5–6.

Прочитать пояснение ↗

.

Открыть обучение и предсказания ↗

```
model = build_model(HIDDEN_LAYERS, ACTIVATION, LEARNING_RATE)

history = model.fit(
    X, y,
    epochs=TRAIN_EPOCHS,
    batch_size=4,
    verbose=0
)
```

- `model = build_model(...)` создаёт **свежую сеть**. Именно поэтому повторный запуск **всей ячейки 6**, а не отдельной строки `fit`, начинает обучение с новых случайных весов.
- `fit(X, y)` обучает на четырёх примерах, сравнивая предсказания с правильными ответами.
- `epochs=500` — 500 проходов по всему набору.
- `batch_size=4` — вся таблица XOR в одном пакете, то есть одна итерация обновления параметров за эпоху.
- `verbose=0` — скрывает длинный текстовый отчёт.
- `shuffle=False` **в этой версии отсутствует**. Для массива `X` перемешивание по умолчанию разрешено, но при полном пакете из четырёх строк порядок почти не влияет на градиент.
- Фиксированного случайного `seed` **тоже нет**, поэтому при разных запусках начальные веса и количество эпох могут различаться.

Что происходит внутри `fit`: прямой проход → расчёт BCE → обратное распространение градиентов → Adam обновляет веса → следующая эпоха. Keras выполняет это автоматически.

Важно: обучение всегда идёт на **тех же четырёх строках**, на которых потом считаем Accuracy. Поэтому итоговая 100% точность показывает, что сеть **выучила таблицу XOR**, но не является независимой проверкой на новых данных.

6. Предсказание, accuracy и эпоха обучения

Ячейка 6, вторая половина.

Открыть результаты и learning_epochs ↗

```
loss, accuracy = model.evaluate(X, y, verbose=0)
probabilities = model.predict(X, verbose=0).ravel()
classes = (probabilities >= 0.5).astype(int)
```

`evaluate` вычисляет функцию потерь и Accuracy. `predict` выдаёт вероятности класса 1. `ravel` делает массив одномерным. Условие `>=0.5` превращает вероятность в правильный/неправильный класс, а `astype(int)` — в 0 или 1.

Accuracy — доля правильно распознанных четырёх комбинаций. Возможные значения: 0%, 25%, 50%, 75%, 100%. При 100% все четыре класса верны. **Loss** при этом не обязан быть нулевым: например, `p=0.51` и `p=0.999` обе дают класс 1, но разную BCE.

Как считается количество эпох

В той же ячейке есть функция `learning_epochs(history_dict)`. Она возвращает **два значения**:

- `first`: первая эпоха, в истории которой Accuracy стала 100%;
- `stable`: первая эпоха, начиная с которой Accuracy оставалась 100% **до последней записанной эпохи**.

Ключевые строки:

```
scores = np.asarray(history_dict["accuracy"], dtype=float)
good = np.isclose(scores, 1.0, rtol=0, atol=1e-8)
first = int(np.flatnonzero(good)[0]) + 1 if np.any(good) else None
stable_here = np.logical_and.accumulate(good[::-1])[::-1]
stable = int(np.flatnonzero(stable_here)[0]) + 1 if np.any(stable_here) else None
```

`good` — массив `True/False` по эпохам; `flatnonzero` находит номера подходящих значений (прибавляем 1, потому что Python считает с нуля). Конструкция с `logical_and.accumulate` проверяет, где **все оставшиеся** значения равны 100%. `None` значит, что нужного события в истории нет.

Мини-пример: Accuracy по эпохам `50%, 75%, 100%, 75%, 100%, 100%`. Тогда `first=3`, `stable=5`. Это **наше условное определение устойчивости только в рамках данного запуска**. Оно не гарантирует правильность на всех будущих эпохах или новых входах.

Если Colab пишет «устойчивая эпоха: 1»: это может означать, что модель уже разделяла все четыре класса на первой эпохе. Но сначала проверь, что запустил **всю ячейку 6** и создал новый объект. При фиксированном пороге Accuracy уже на старте может оказаться 100%, даже когда вероятности ещё не близки к целевым значениям.

Тонкость сравнения: Keras считает тренировочную Accuracy во время прохода по пакету, а собственная версия ниже записывает Accuracy **после обновления параметров**. Поэтому номера эпох этих двух реализаций не стоит напрямую сопоставлять как строго одинаковые измерения.

7. Ручной расчёт ошибок: MSE и BCE

Ячейки 7–8.

Открыть формулы ↗

.

Открыть расчёт в коде ↗

MSE (среднеквадратичная ошибка): `MSE = mean((y − p)²)`. Например, для правильного `y=1` и вероятности `p=0.8` ошибка равна `(1−0.8)²=0.04`.

BCE (бинарная кросс-энтропия):

`BCE = −mean(y·ln(p) + (1−y)·ln(1−p))`.

Например, если правильный ответ 1, то `BCE = −ln(p)`: при `p=0.9` это ≈0.105, а при `p=0.1` ≈2.303. Неверная уверенная вероятность наказывается гораздо сильнее.

```
p = np.clip(probabilities.astype(np.float64), 1e-7, 1 - 1e-7)
per_example_mse = (y - p) ** 2
per_example_bce = -(y * np.log(p) + (1 - y) * np.log(1 - p))
print(per_example_mse.mean())
print(per_example_bce.mean())
```

`np.clip` ограничивает вероятности и предотвращает попытку вычислить `ln(0)`. `mean()` усредняет ошибки четырёх примеров.

Зачем две функции? Это дополнительная учебная проверка расчёта ошибки. **Обучается Keras-модель по BCE**, MSE только вычисляется после обучения. Если преподаватель спрашивает «что мы минимизируем?», отвечай: **BCE**.

8. Графики основной сети

Ячейки 9–10.

Прочитать, что показывают графики ↗

.

Открыть построение графиков ↗

```
epochs = np.arange(1, len(history.history["loss"]) + 1)
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot(epochs, history.history["loss"])
axes[1].plot(epochs, history.history["accuracy"])
```

`epochs` — номера эпох от 1 до 500. `plt.subplots(1,2)` создаёт **два графика в одной строке**. `axes[0]` — Loss; `axes[1]` — Accuracy. `axes[i].set(...)` задаёт заголовки и подписи. `ylim=(-0.05,1.05)` оставляет запас вокруг верхней границы 1. `tight_layout` подгоняет отступы, `show` отображает рисунок.

- **Loss снижается** — вероятности приближаются к правильным меткам.
- **Accuracy растёт ступенями** — примеров лишь четыре; каждый правильно классифицированный пример добавляет 25 процентных пунктов.
- Вертикальная **пунктирная линия** (`axvline(stable_epoch, ...)`) показывает нашу эпоху устойчивого достижения 100%. Линия появляется, только если `stable_epoch is not None`.

Не путай: запланированные 500 эпох — длина графика; `stable_epoch` — метка момента, начиная с которого все дальнейшие записи Accuracy равны 100%. Loss может продолжать уменьшаться и после этой линии.

Что сказать вслух: «Слева показана динамика бинарной кросс-энтропии, справа — доля правильных классификаций. Пунктир отмечает эпоху устойчивого решения всех четырёх случаев XOR в данном запуске».

9. Сравнение архитектур: что именно меняем

Ячейки 11–12.
Открыть план экспериментов ↗
.
Открыть цикл сравнения ↗

Здесь ровно три дополнительных Keras-эксперимента. Вместе с основной сетью `2–4–1 tanh` из раздела 3 блокнот показывает **четыре конфигурации в сумме** — это важно при ответе на вопрос «сколько сетей в работе?».

Конфигурация	Скрытые слои	Активации	Для чего
Основная 2–4–1	один слой, 4	`tanh` → `sigmoid`	Базовая обучаемая модель
Контроль 2–1	нет	только выход `sigmoid`	Показать ограничение линейной границы
Эксперимент 2–4–1	один слой, 4	`sigmoid` → `sigmoid`	Сравнить с основной при той же ширине
Собственная 2–6–4–1	два слоя: 6 и 4	`tanh`, `tanh` → `sigmoid`	Проверить влияние глубины и размера

В цикле `for name, hidden, activation, reason in experiments:` для каждой конфигурации вызываются `build_model`, `fit` и `learning_epochs`. Затем печатаются количество скрытых слоёв, число нейронов, активации, объяснение выбора и эпоха. **Сразу после этого строится отдельная пара графиков Loss и Accuracy для каждого варианта.** В конце выводится краткое сравнение эпох.

Важно про контроль 2–1: параметр `hidden=()` не создаёт ни одного скрытого слоя; указанный в записи эксперимента `tanh` в действительности **не применяется**, потому что цикл по скрытым слоям пуст.

Параметры Keras в цикле общие: таблица XOR, `TRAIN_EPOCHS=500`, `batch_size=4`, `BCE`, `Adam(lr=0.05)`, порог Accuracy 100%. Начальные веса случайные; без фиксированного seed результат может меняться. Это единый бюджет эксперимента, но сравнение на одном запуске не доказывает, что какая-то архитектура всегда лучше.

Как объяснить каждый эксперимент

Контроль 2–1. Один выходной нейрон без скрытых нелинейных преобразований не может идеально разделить XOR. Если сеть не достигла 100% до окончания лимита, выводим «не обучилась за 500 эпох», а не «вообще никогда не обучится» (строже: для этой архитектуры XOR математически неразделим).

2–4–1, sigmoid. Та же ширина, что у основной `2–4–1 tanh`, но другая функция скрытого слоя. Это наиболее чистая из наших пар для сравнения влияния активации — хотя случайная инициализация всё равно влияет.

2–6–4–1, tanh. Два скрытых слоя из 6 и 4 нейронов — наш эксперимент с более глубокой архитектурой. Проверяем, изменится ли число эпох. Для XOR такая глубина **не обязательна**, она не объявляется «лучшей» заранее.

10. Обоснование выбора для защиты

Ячейка 13.

Открыть готовые выводы ↗

Самая сильная короткая логика обоснования:

1. **Два входа** — потому что XOR получает два бита.
2. **Один выход с `sigmoid`** — потому что целевой класс один из двух (0/1), и нам удобна вероятность класса 1.
3. **Скрытый нелинейный слой** — потому что XOR линейно неразделима.
4. **4 нейрона и `tanh` в основной модели** — простой и наглядный рабочий выбор, а не найденный глобальный оптимум.
5. **BCE** — функция потерь для бинарной вероятностной классификации.
6. **Adam** — удобный оптимизатор, адаптирующий обновления весов. Скорость 0.05 — выбранный для учебного эксперимента гиперпараметр.
7. **Другие архитектуры** — требование сравнить количество нейронов, слоёв и активации, а не желание усложнить программу «для красоты».
8. **Эпохи и графики** — основа сравнения; если Accuracy всех обучившихся равна 100%, сама итоговая Accuracy не различает скорость обучения.

Сколько весов у 2–6–4–1? Переход `2→6`: `2×6+6=18`; `6→4`: `6×4+4=28`; `4→1`: `4×1+1=5`. Итого **51 обучаемый параметр**, то есть **три обучаемых Dense-слоя**. «Два скрытых слоя» и «три обучаемых слоя» не противоречат друг другу: третий — выходной.

Чего НЕ утверждаем: что глубокая сеть всегда лучше, что ReLU участвовала в нашем текущем сравнении, что одна архитектура статистически победила по единственному запуску, что 100% Accuracy на тренировочных четырёх точках означает обобщение на новые данные.

11. ООП-реализация: зачем и как устроен класс

Ячейки 14–15.

Прочитать постановку ООП-задачи ↗

.

Открыть класс NeuralNetwork ↗

В этом разделе **Keras вообще не используется**: нейронная сеть написана как Python-класс `NeuralNetwork`, а NumPy выполняет матричные операции. Это показывает, как работает сеть внутри, а не только как вызвать готовую библиотеку.

Архитектура фиксирована: **2–4–1**, скрытая активация `tanh`, выходная `sigmoid`. В отличие от функции `build_model`, этот класс **не умеет произвольно менять архитектуру** — и это специально, чтобы код было проще разбирать.

Конструктор `__init__`:

```
self.w1 = np.random.randn(2, 4) * np.sqrt(2 / (2 + 4))
self.b1 = np.zeros((1, 4))
self.w2 = np.random.randn(4, 1) * np.sqrt(2 / (4 + 1))
self.b2 = np.zeros((1, 1))
```

`self` означает текущий экземпляр класса. `w1` — матрица 2×4 для перехода «вход → скрытый», `b1` — четыре смещения; `w2` — матрица 4×1 «скрытый → выход», `b2` — одно смещение. Веса сначала случайные, смещения нулевые. Небольшое масштабирование случайных весов облегчает старт обучения.

Метод `sigmoid`:

```
def sigmoid(self, z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))
```

`np.clip` здесь защищает вычисление экспоненты от чрезмерно больших по модулю аргументов. Это числовая защита, не новая нейронная архитектура.

12. Как собственная сеть делает прямой проход

Ячейка 15, метод `forward`.
Открыть forward ↗

```
def forward(self, X):
    hidden = np.tanh(X @ self.w1 + self.b1)
    output = self.sigmoid(hidden @ self.w2 + self.b2)
    return hidden, output
```

Знак `@` — **матричное умножение**.

При подаче всей таблицы `X` размера **4×2**:

1. `X @ w1`: `(4×2) @ (2×4) → 4×4` — значения скрытых нейронов для каждого примера;
2. `+ b1`: прибавляем четыре смещения к каждой строке;
3. `np.tanh`: нелинейные скрытые активации, матрица остаётся 4×4;
4. `hidden @ w2`: `(4×4) @ (4×1) → 4×1`;
5. `+ b2`, затем `sigmoid`: четыре выходные вероятности, по одной на строку.

Мы возвращаем **оба** результата, поскольку `hidden` понадобится при вычислении градиентов. Метод `predict` берёт вероятности из `forward(X)[1]` и применяет порог `>= 0.5`.

13. Обратное распространение и простой градиентный спуск

Ячейка 15, метод `fit`.

Открыть цикл обучения класса ↗

```
def fit(self, X, y, epochs=2500, lr=0.5):
    y = np.asarray(y).reshape(-1, 1)
    history = {"loss": [], "accuracy": []}

    for _ in range(epochs):
        hidden, output = self.forward(X)
        delta2 = (output - y) / len(X)
        dw2 = hidden.T @ delta2
        db2 = np.sum(delta2, axis=0, keepdims=True)
        delta1 = (delta2 @ self.w2.T) * (1 - hidden**2)
        dw1 = X.T @ delta1
        db1 = np.sum(delta1, axis=0, keepdims=True)
        self.w1 -= lr * dw1
        self.b1 -= lr * db1
        self.w2 -= lr * dw2
        self.b2 -= lr * db2
```

Пошагово:

1. `y.reshape(-1,1)` делает правильные ответы столбцом 4×1, как и предсказания `output`.
2. `for` выполняет указанные `epochs=2500` итераций; одна итерация обрабатывает все четыре примера и делает одно обновление параметров.
3. `delta2 = (output-y)/len(X)` — удобное упрощение производной **BCE + sigmoid** на выходе; делим на 4, потому что ошибка усредняется по четырём наблюдениям.
4. `dw2 = hidden.T @ delta2` и `db2 = sum(delta2)` вычисляют градиенты выходного слоя.
5. `delta1 = (delta2 @ w2.T) * (1-hidden**2)` распространяет ошибку назад через скрытый слой. Множитель `1-hidden**2` — производная `tanh`, когда `hidden = tanh(z)`.
6. `dw1`, `db1` — градиенты первого обучаемого слоя.
7. `w -= lr * dw` и `b -= lr * db` — **обычный градиентный спуск**, то есть изменение весов и смещений в направлении уменьшения ошибки.

Здесь **не Adam**: нет накопительных моментов градиентов. Это существенно облегчает защиту. `lr=0.5` отличается от Keras `0.05`, поэтому прямой рейтинг «ООП быстрее Keras» некорректен.

После изменения весов выполняются повторный `forward`, расчёт BCE и Accuracy; оба числа добавляются в `history`. Следовательно, для ООП-класса история записывается по состоянию **после каждого обновления**.

14. Как запускаем ООП-сеть и строим её графики

Ячейки 16–19.
Открыть запуск ООП ↗
.
Открыть графики ООП ↗

```
network_oop = NeuralNetwork()
history_oop = network_oop.fit(X, y, epochs=2500, lr=0.5)

p_oop = network_oop.forward(X)[1].ravel()
pred_oop = network_oop.predict(X).ravel()
```

Создаётся **новый объект** с новыми случайными весами. Затем он учится на той же таблице. Выводятся архитектура, оптимизатор, правильные и предсказанные классы, итоговая BCE и Accuracy.

```
_, oop_stable = learning_epochs(history_oop)
```

Мы используем **ту же** ранее объявленную функцию `learning_epochs`, но передаём словарь `history_oop` (у Keras история лежит в `history.history`). Подчёркивание означает «первое значение вернулось, но сейчас оно не нужно».

Ячейка 19 строит две кривые из `history_oop["loss"]` и `history_oop["accuracy"]` по 2500 эпохам. Если получено значение `oop_stable`, рисуется пунктир. Графики выполняют то же требование, что и графики Keras, но для собственной реализации.

Что на защите: «В первой части Keras сам вычисляет и обновляет веса при помощи Adam; здесь я вручную реализовал прямой проход, обратное распространение и обычный градиентный спуск в Python-классе».

15. Как показывать работу на защите

Ячейка 20 содержит краткий план выступления.
Открыть шпаргалку ↗

Порядок демонстрации:

1. Открой таблицу XOR (**ячейка 1**) и скажи, почему без нелинейного скрытого слоя задача не решается идеально.
2. Покажи базовую архитектуру и её настройки (**ячейки 3–4**): 2 входа, 4 `tanh`, 1 `sigmoid`, BCE, Adam.
3. Покажи предсказания и число эпох (**ячейка 6**). Объясни разницу между запланированными 500 эпохами и `stable_epoch`.
4. Покажи две кривые основной модели (**ячейка 10**).
5. Пройди по трём экспериментам (**ячейка 12**): у каждого архитектура, активация, пара графиков и эпоха успешного обучения или её отсутствие.

6. Покажи свой простой класс (**ячейка 15**), особенно `forward`, `delta2`, `delta1` и обновления `w -= lr * dw`.
7. Покажи результаты и графики собственной сети (**ячейки 17 и 19**).

Пять вопросов, которые стоит отрепетировать

Почему нужен скрытый слой? XOR линейно неразделима; нелинейный скрытый слой позволяет построить необходимую границу.

Почему `sigmoid` на выходе? Нам требуется вероятность одного из двух классов; порог 0,5 даёт предсказанный 0 или 1.

Почему BCE, а не только Accuracy? Accuracy оценивает уже пороговые классы, а BCE чувствительна к величинам вероятностей и подходит для оптимизации.

Почему именно 2–6–4–1? Чтобы проверить влияние второго скрытого слоя. Это экспериментальный более глубокий вариант, а не заведомо оптимальная архитектура XOR.

Почему числа эпох разнятся при перезапуске? Начальные веса случайные и в коде нет фиксированного seed; достижение 100% зависит от траектории обучения. Кроме того, Keras и ООП имеют разные оптимизаторы и скорости обучения.

16. Что именно мы взяли из книги, а что добавили сами

Книга: Крис Элбон, *«Машинное обучение с использованием Python. Сборник рецептов»*, БХВ-Петербург, 2019, глава 20 «Нейронные сети».

Раздел книги	Где искать	Что применили в работе
20.1. Предобработка данных	стр. 318–319	Обсуждение необходимости общего масштаба признаков; XOR уже бинарен
20.2. Проектирование нейронной сети	стр. 320–322	`Sequential`, `Dense`, скрытые и выходные слои, функции активации
20.3. Тренировка бинарного классификатора	стр. 323–324	`compile`, `fit`, бинарная классификация, метрика точности
20.6. Выполнение предсказаний	стр. 329–330	`predict` для получения вероятностей класса
20.7. Визуализация истории тренировки	стр. 331–333	Графики Loss и Accuracy на основе `history`

В примерах книги бинарная классификация рассматривается, в частности, на **отзывах о фильмах**, а не на XOR. Книга использует более старую версию Keras и местами другой оптимизатор, например `rmsprop`. Мы перенесли **подход**, а не копировали в точности её исходные наборы данных и конфигурации.

То, что было нашим выбором по заданию: сама задача XOR; конкретные архитектуры 2–1, 2–4–1 и 2–6–4–1; `tanh` для базовой и глубокой модели; Adam с шагом 0.05; функция `learning_epochs`; отдельная ООП-реализация через NumPy и простой градиентный спуск. Конкретные числа эпох не даны книгой: их выводит твоя программа во время запуска.

Итоговое объяснение одной минутой

«Я решал задачу XOR на Python двумя способами. Сначала сделал полносвязную сеть Keras с архитектурой 2–4–1: два бинарных входа, четыре скрытых нейрона `tanh` и один выход `sigmoid`. Обучал её по бинарной кросс-энтропии оптимизатором Adam. Показал четыре предсказания, графики Loss и Accuracy и определил эпоху, с которой точность на всех четырёх комбинациях оставалась 100% до конца запуска. Затем сравнил три дополнительные архитектуры: контрольную 2–1 без скрытого слоя, 2–4–1 с `sigmoid` и собственную более глубокую 2–6–4–1 с `tanh`. Наконец, написал отдельный класс `NeuralNetwork` с фиксированной сетью 2–4–1, где самостоятельно реализовал матричный прямой проход, обратное распространение и обычный градиентный спуск. Главный вывод: для XOR нужен нелинейный скрытый слой, а более сложная архитектура не обязательно обучается быстрее».

Практический совет: перед защитой сделай один полный запуск **«Среда выполнения → Выполнить все»** и используй **эпохи и графики именно этого запуска**, а не старые числа из чужой проверки.

Основа: ваш последний блокнот XOR_Colab_простая_ООП.ipynb и К. Элбон, «Машинное обучение с использованием Python», глава 20 (2019). Данные реального запуска нужно брать из вашего Colab; здесь не подставляются вымышленные эпохи.